

# Interactive Image Mosaic Generator

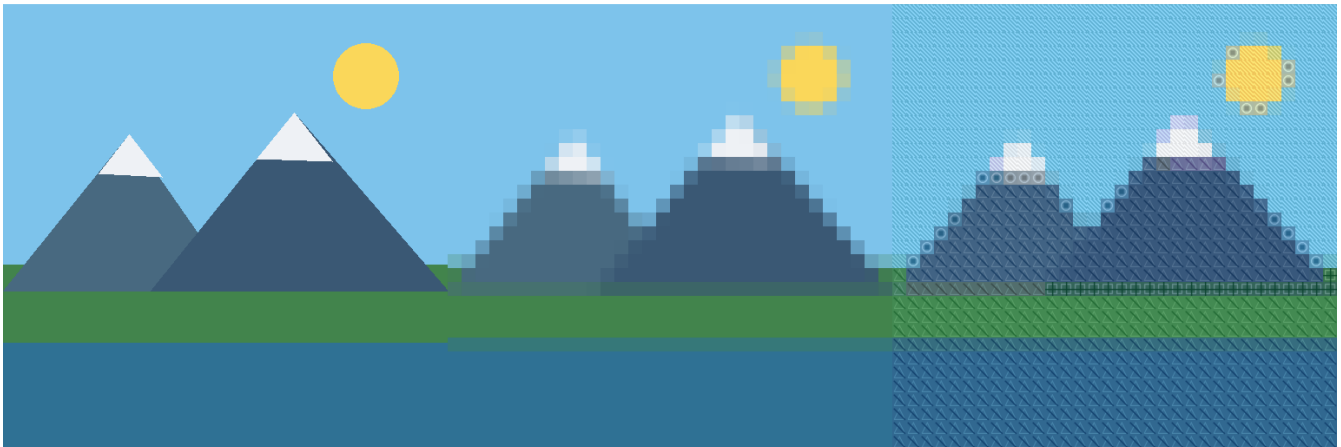
## ***Performance Report***

**Approach.** Images are center-cropped and resized to 512 x 512. The user selects a 16 x 16, 32 x 32, or 64 x 64 grid. Mean RGB values are computed using vectorized NumPy, with a nested-loop implementation included for comparison.

Three deterministic tile sets - Geometric, Dots, and Stripes - contain 24 mini-images each. Cells are matched to the closest tile by squared Euclidean distance between mean RGB values. Natural, Colorized, and High Contrast rendering styles are supported.

The interface shows the original/preprocessed image, segmented color grid, and final mosaic, and reports MSE, SSIM, processing time, and color-category counts.

## **Reference output**



**Pipeline:** preprocess -> segment -> vectorized analysis -> color classification -> tile matching -> reconstruction -> MSE/SSIM -> benchmark.

## Performance comparison

| Grid    | Cells | Vectorized (ms) | Nested loop (ms) | Speedup |
|---------|-------|-----------------|------------------|---------|
| 16 x 16 | 256   | 4.512           | 5.635            | 1.25x   |
| 32 x 32 | 1,024 | 4.645           | 8.583            | 1.85x   |
| 64 x 64 | 4,096 | 5.335           | 21.108           | 3.96x   |

The loop implementation scales more sharply because Python processes every cell separately. The vectorized implementation performs the reduction in optimized NumPy array code, so the performance advantage grows with grid resolution.

At 64 x 64, the reference benchmark shows vectorized analysis at 5.335 ms versus 21.108 ms for nested loops, a 3.96x speedup.

## Reconstruction quality

For the included landscape example at 32 x 32 using Geometric tiles and Colorized style, one reference run produced **MSE 439.40**, **SSIM 0.2101**, and approximately **0.16 s** processing time.

## Verification

Automated tests verify preprocessing, all grid sizes, vectorized/loop equivalence, tile mapping, rendering styles, MSE/SSIM validity, color-category accounting, example generation, and end-to-end processing. GitHub Actions verifies the repository on every push.

## Conclusion

The project completes the required pipeline from preprocessing and segmentation through tile matching, reconstruction, similarity evaluation, interactive presentation, and reproducible performance analysis.